TP6 – TDD: Documento de decisiones de diseño

User Story: Inscribirme a actividad (EcoHarmony Park) · **Grupo 8** · ISW 4K3 · UTN FRC · 2026
Estado: v1.0 · 2026-10-15
Documentos hermanos: plan de pruebas · preguntas al PO · partición de equivalencias · código

> COMO visitante QUIERO inscribirme a una actividad PARA reservar mi lugar en la misma.

La consigna del práctico pide, además del código y de las pruebas, un *documento que justifique las decisiones de diseño tomadas*. Este es ese documento. Cada decisión se presenta con el mismo formato: **qué se decidió**, **por qué**, **qué alternativa se descartó** y **qué prueba lo respalda**, para que se pueda verificar en el código.

---

1. Resumen

Tema	Resultado
Lenguaje y estilo	Python 3.13 (backend) y JavaScript/React 19 (front). **PEP 8**, verificado automáticamente con `ruff` (largo de línea 100).
Enfoque	TDD de Kent Beck: lista de pruebas, y luego un ciclo **Red** → **Green** → **Refactor** por cada comportamiento, visible en `git log`.
Pruebas automatizadas	**318 del backend** (283 sin base de datos + 35 contra PostgreSQL real y `qrcode` real) y **108 del front**. Todas verdes.
Persistencia	PostgreSQL 16 en Docker, SQL plano (sin ORM), visible desde DBeaver.
Piezas reales y gratuitas	QR con la librería `qrcode`; email por SMTP genérico (Gmail o Brevo); PostgreSQL local.
Interfaz	React + Vite, paleta y tipografía (Lato) del PO, estilo *Liquid Glass*, responsive.

Todo lo que el PO puede cambiar (horarios, cupos, edades, talles, máximo por inscripción) es **configuración inyectada**, no lógica fija. Las respuestas del PO (preguntas P01–P32) se absorbieron cambiando datos, no reescribiendo pruebas.

2. Cómo se aplicó TDD

2.1 Proceso

1. **Test List** (plan de pruebas): antes de escribir código se listaron las pruebas, de la más simple a la más compleja, y se enviaron las dudas al PO. La lista no se consideró cerrada: las pruebas que fueron apareciendo se anotaron en ella antes de escribirse.

2. **Un ciclo por comportamiento**, siempre en el mismo orden y con un commit por cada etapa:

- `test(ID): rojo` – la prueba nueva **falla por la razón correcta** (se verifica antes de commitear).
- `feat(ID): verde` – el código mínimo para que pase (*Fake It*, *Triangulate* u *Obvious Implementation*).
- `refactor(...)` – mejora del diseño sin agregar comportamiento y con todo en verde. 3. **Historial agrupado.** Para que el historial sea legible no hay un commit por cada prueba: se agrupan las pruebas de un mismo tema en un commit rojo y un commit verde (*Clean Check-in* de Beck). Cuando una prueba pasa de entrada se lo dice en el mensaje (`ya verde`). 4. **Prueba de aceptación primero** (ATDD, Ble cap. 3): `ACE-01` se escribió antes que el servicio, marcada como *fallo esperado*, y se activó cuando las piezas estuvieron.

2.2 Patrones de Beck usados y dónde se ven

Patrón	Dónde
Starter Test	Primer ciclo: `Actividad` con cupo 10 y pedido de 3 → quedan 7 (`ACT-01`).
Fake It / Triangulate	Cupos (`ACT`) y edades mínimas: se parte de una constante y se generaliza al agregar un segundo caso.
Obvious Implementation	Validaciones simples de `Visitante` (DNI, edad, nombre).
Child Test	`INS-*` se dividió en pruebas chicas cuando una combinación de reglas falló.
Mock Object	`ServicioEmail` (acción): se verifica que se envió una vez con el mensaje correcto.
Crash Test Dummy	Dobles que lanzan excepción: email caído, QR roto, repositorio caído (`SRV-15..17`).
Clean Check-in	Nada se commitea en rojo salvo la prueba nueva y nada se commitea sin ejecutar la suite.
Assert First y *Evident Data*	Las pruebas empiezan por el `assert` ; los datos se leen a simple vista (cupo 10, piden 3).

2.3 Política de dobles de prueba (Ble, cap. 6)

Colaborador	Tipo	Doble en unitarias
`Reloj` , `Parque` , lecturas del repositorio, `GeneradorQR`	consulta	*stub / fake*: se verifica el **estado**, no cuántas veces se llamó
`ServicioEmail`	acción	*mock*: se verifica que se envió, una vez, con el mensaje correcto
Repositorios	almacenamiento	*fake* en memoria **validado con las mismas pruebas de contrato que el real**
Fallas (email, QR, base)	error improbable	*Crash Test Dummy*

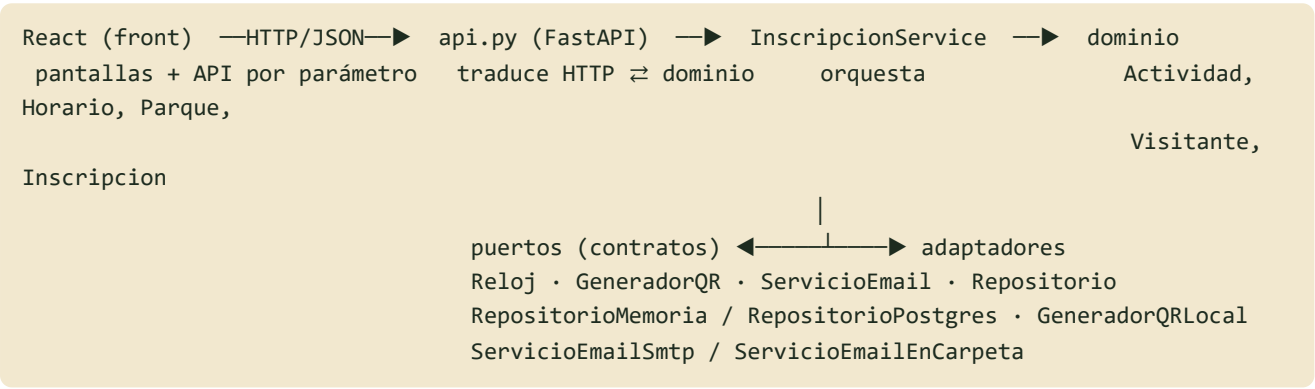
Por qué. Verificar llamadas a una consulta vuelve la prueba frágil (se rompe ante un refactor que no cambia el comportamiento). Y un fake que no se contrasta con el real termina mintiendo: por eso existe el grupo de **pruebas de contrato**, que se ejecutan dos veces, contra el fake y contra PostgreSQL (§6.2).

2.4 Categorías de pruebas

Cat.	Qué usa	Cuántas	Cuándo corre
U – unitarias	dominio y dobles; sin red, disco ni reloj real	283 (backend) + 108 (front)	siempre (`pytest` , `vitest`)
C – contrato / integración local	PostgreSQL local, `qrcode`	35	con la base levantada (marcadores `db` y `local`); sin base se saltean con un aviso
X – externas	email real a una casilla	pendiente (ver §11)	manual, antes de la entrega (marcador `externa`)

3. Arquitectura

3.1 Capas



Capa	Módulos (`backend/src/ecoharmony/`)	Responsabilidad
Dominio	`actividad` , `horario` , `parque` , `visitante` , `inscripcion` , `motivos` , `terminos` , `terminos_texto` , `catalogo` , `usuarios` , `contrasenas`	Reglas del negocio. Sin I/O.
Aplicación	`inscripcion_service` , `disponibilidad` , `mensaje_email` , `qr`	Orquesta el caso de uso; armar el mensaje y el contenido del QR son funciones puras.
Puertos	`puertos`	Contratos (`Protocol`) de reloj, QR, email y repositorios.
Adaptadores	`repositorio_memoria` , `repositorio_postgres` , `repositorio_usuarios_*` , `generador_qr_local` , `servicio_email_smtp` , `servicio_email_en_carpeta` , `reloj`	Lo que toca el mundo exterior.
Entrada	`api` , `main` , `preparar_base` , `entorno`	HTTP, armado de dependencias, configuración.

Decisión: dependencias hacia adentro e inyección por constructor. `InscripcionService` recibe repositorio, parque, reloj, generador de QR y servicio de email. Nada del dominio importa un adaptador. **Por qué:** es lo que permite el TDD sin infraestructura (Ble: dobles) y reemplazar un doble por la pieza real sin tocar `Inscripcion` ni el servicio. **Descartado:** instanciar los colaboradores dentro de las clases (haría imposible simular la hora, un email caído o una base caída).

3.2 Modelo del dominio

Clase	Rol
`Actividad`	Nombre, cupo por turno, edad mínima, si requiere talla y sus T&C. Dueña de la regla de cupos.
`Horario`	Fecha + hora de inicio de un turno de una hora.
`Parque`	Días y horas de funcionamiento (lunes, 25/12 y 1/1 cerrado; 8:30 a 19:00).
`Visitante`	Nombre, DNI, edad y talla; se valida a sí mismo contra la actividad.
`Inscripcion`	Reúne titular, actividad, horario, visitantes y T&C aceptados; aplica las reglas combinadas.
`Usuario`	Email y contraseña; el visitante debe estar registrado (decisión del grupo).
`TerminosYCondiciones`	Actividad, versión y texto.
`MotivoRechazo`	Enumeración de las razones por las que se rechaza (§4.1).

4. Decisiones de diseño

D1. Rechazar con un motivo, no con un booleano

Se decidió: `Inscripcion` y el servicio devuelven una lista de `MotivoRechazo` (`SIN_CUPO`, `DNI_INVALIDO`, `TALLA_REQUERIDA`, `PARQUE_CERRADO`, `FUERA_DE_VENTANA`, `TERMINOS_NO_ACEPTADOS`, `DNI_YA_INSCRIPTO`, `USUARIO_NO_REGISTRADO`, …) y, para los visitantes, un diccionario *posición → motivos*.
Por qué: una prueba que solo mira `False` no distingue un rechazo correcto de uno accidental. Con el motivo, cada prueba verifica *la razón*. Además es lo que permite al front mostrar "Visitante 1: el DNI debe tener 7 u 8 dígitos".
Descartado: lanzar una excepción por regla (obliga a atrapar de a una y se pierden los demás errores) o devolver un texto (las pruebas quedarían atadas al idioma).
Respaldo: `INS-01..20`, `VIS-01..18`, `SRV-*`, `MSJ-01..07` (front).

D2. Todas las reglas del PO como configuración inyectada

Se decidió: cupos, edades mínimas, tallas, máximo de 10 personas, ventana de 24 h a 15 min y horarios del parque son datos (constantes con nombre, `catalogo.py`, constructor de `Parque`), no condiciones sueltas en la lógica.
Por qué: el PO ya cambió respuestas durante el trabajo y el grupo podía equivocarse en una; cambia un dato, no una prueba. Los textos de T&C también se arman con esos datos (D12), de modo que no hay números repetidos que se desactualicen.
Respaldo: `ACT-*`, `PAR-*`, `HOR-*`, `CAT-*`.

D3. El tiempo es un colaborador (`Reloj`)

Se decidió: el servicio recibe un `Reloj` y nunca consulta la hora del sistema.
Por qué: "ya comenzó", "falta menos de 15 minutos" y "más de 24 horas" no se pueden probar con el reloj real. Las pruebas fijan el reloj en el miércoles 2026-11-04 08:00.
Respaldo: `HOR-*`, `INS-*`, `test_reloj.py`.

D4. Un QR por visitante, en un único email

Se decidió: el contenido del QR se arma con una función pura (`contenido_del_qr`: código de reserva, actividad, horario, DNI) y se genera un PNG por visitante, todos adjuntos a **un** email.
Por qué: lo confirmó el PO (cada persona entra con su propio QR); un solo email evita que el titular reciba un mensaje por visitante. Separar *qué dice el QR* de *cómo se dibuja* permite probar el contenido sin imagen y reemplazar la librería sin tocar nada más.
Respaldo: `QR-01..08` (el contrato se ejecuta con el generador falso y con `qrcode`, que devuelve un PNG válido y distinto por contenido).

D5. Reserva de cupo atómica en SQL y restricción de unicidad en la base

Se decidió: el cupo se reserva con **una sola sentencia** `INSERT … ON CONFLICT DO UPDATE … WHERE ocupados + n <= cupo`, y el conflicto de horario por DNI se refuerza con `UNIQUE (dni, fecha, hora)`. El servicio traduce la carrera a `DNI_YA_INSCRIPTO`.
Por qué: "leer el cupo, comparar y escribir" son tres pasos: dos pedidos simultáneos por el último lugar ganarían los dos. La comprobación previa del servicio (`SRV-13`) da el mensaje amable; la restricción de la base es la garantía real.
Descartado: bloquear en la aplicación (no sirve con más de un proceso) y confiar solo en la comprobación previa.
Respaldo: `REP-05` (dos hilos disputando el último lugar contra PostgreSQL real), `REP-09`, `SRV-19`, `SRV-20`.

D6. Orden de las operaciones al inscribir

Se decidió: validar → generar el código de reserva → **generar los QR** → **guardar** (reserva de cupo + inscripción) → **enviar el email con reintentos**. Si el email no sale después de 3 intentos, se **anula** la inscripción (libera cupo y DNI) y se informa el error (503).
Por qué:

- El PO dijo que el email no debería fallar y el grupo decidió que, si falla, **la inscripción no queda confirmada** (P13b). Reintentar antes de anular cubre las caídas transitorias.
- Se generan los QR *antes* de guardar para que un QR roto no deje una inscripción a medias (`SRV-16`).

Limitación conocida (honestidad del diseño): si el proceso se cae justo entre guardar y enviar, la inscripción quedaría guardada sin email. La mejora natural es un estado *pendiente* con reintentos en segundo plano; se dejó fuera del alcance de este TP.
Respaldo: `SRV-01..17`.

D7. Validar sin efectos (`POST /inscripciones/validar`)

Se decidió: el servicio tiene `validar` además de `inscribir`: aplica las mismas reglas pero **no guarda, no reserva cupo ni envía nada**. La API lo expone y el paso 3 del front lo usa al tocar *Continuar*.
Por qué: el grupo pidió que los errores de los visitantes (por ejemplo "Visitante 1: el DNI debe tener 7 u 8 dígitos, la edad debe estar entre 0 y 100") aparezcan en el paso donde se cargan y no recién en la confirmación. La alternativa obvia, repetir las reglas en JavaScript, tiene dos fuentes de verdad que terminan divergiendo. Con `validar`, **las reglas viven en un solo lugar** y el front solo traduce motivos a mensajes y campos.
Efecto colateral útil: el mismo chequeo incluye cupo y DNI ya inscriptos, así que esos avisos también

llegan antes.
Respaldo: `SRV-24..30`, `API-14..17`, `VAL-F01..14`, `CAM-01..04`, `APP-10..12`.

D8. Persistencia: PostgreSQL con SQL plano

Se decidió: PostgreSQL 16 (Docker, puerto 5433), `psycopg` 3 y SQL escrito a mano (`sql/schema.sql`), sin ORM. Dos bases: `ecoharmony` (desarrollo) y `ecoharmony_test` (pruebas). El esquema se escribe con `CREATE … IF NOT EXISTS`, de modo que `preparar_base` se puede volver a ejecutar.
Por qué: el grupo quería abrir la base en DBeaver; el SQL plano se ve y se entiende (el grupo eligió "el SQL más simple" y PEP 8). El esquema reaplicable permite agregar tablas nuevas a una base ya existente sin perder datos (`PREP-03`).
Descartado: SQLite (no se ve igual en DBeaver ni tiene el mismo manejo de concurrencia) y un ORM (oculta justo la sentencia atómica de D5).
Respaldo: `test_esquema_postgres.py`, `test_preparar_base.py`, `REP-*`.

D9. Fake en memoria + prueba de contrato

Se decidió: el mismo conjunto de pruebas (`REP-01..09`) se ejecuta contra `RepositorioMemoria` y contra `RepositorioPostgres`.
Por qué: el fake sirve para que las unitarias sean rápidas, pero solo si se comporta como el real. **La prueba de contrato ya encontró un defecto real:** en el fake, `guardar` reservaba el cupo sobre la *copia* de la actividad que traía la inscripción y no sobre la del repositorio (`REP-03`). Sin el contrato, el fake habría mentido.

D10. Usuarios registrados y sesión simple

Se decidió: el visitante debe registrarse con **email + contraseña**; el email de la inscripción es el del usuario (P01/P02). Contraseñas con `hashlib.scrypt` y sal (biblioteca estándar); sesión como token firmado con HMAC-SHA256 que vence a las 8 horas. Email y contraseña incorrectos fallan con el mismo mensaje.
Por qué: la US no decía a qué email enviar; unir inscripción con usuario resuelve la pregunta y evita pedir un email por cada inscripción. Se usó solo la biblioteca estándar para no sumar dependencias por algo simple.
Privacidad: sin sesión, el rechazo es siempre `USUARIO_NO_REGISTRADO` y **no** revela si un DNI ya estaba inscripto (`SRV-23`, hallazgo de la prueba manual).
Descartado: guardar la contraseña con un hash rápido (SHA) o en texto plano; JWT con librería externa.
Respaldo: `USU-01..09`, `INS-20`, `SRV-21`, `SRV-23`, `ACE-08`.

D11. Email: un adaptador SMTP genérico y un reemplazo para desarrollo

Se decidió: `ServicioEmailSmtp` habla SMTP con STARTTLS y un **tiempo límite de 10 s**; sirve para Gmail o Brevo cambiando solo el `.env`. Si el `.env` no tiene SMTP, la API arranca con `ServicioEmailEnCarpeta`, que guarda el texto y los QR en `salida_de_emails/` (ignorada por git).
Por qué: el PO exige un email real, pero las pruebas no pueden depender de internet ni de credenciales, y el grupo no quiere comprometerse con un proveedor. La carpeta de desarrollo permite probar todo el recorrido sin enviar nada.
Hallazgo de la prueba manual: con credenciales SMTP de ejemplo en el `.env`, la API se quedaba colgada intentando conectarse. Se agregó el tiempo límite (`SMTP-06`) y el `.env.example` trae el SMTP comentado.

Respaldo: `SMTP-01..06`, `MAIL-01/02`, `EML-01..07`; la prueba externa con un email real está pendiente (§11).

D12. Términos y condiciones propios de cada actividad

Se decidió: un texto distinto por actividad, redactado por `redactar_terminos(nombre, edad_minima, requiere_talla, cupo_por_turno)` en `terminos_texto.py`. Las reglas (edad, talla, cupo, 24 h, 15 min, máximo de personas) salen de los mismos datos que usa el dominio; lo propio de cada actividad (seguridad de la tirolesa, trato de los animales en el safari, arnés de la palestra, herramientas de jardinería) es texto fijo. Llevan versión (`v1`) y se aceptan por actividad.
Por qué: el PO pidió un texto por actividad, diseñado por el grupo y sin validación de su parte. Armarlos con los datos evita que el texto diga "8 años" mientras la regla dice 10. Una actividad sin texto propio **no se puede crear** (error), para que nunca salga una sin T&C.
Importante: el contenido es un **borrador del grupo para revisar** (puntos a confirmar en §11).
Respaldo: `TYC-01..13`, `CAT-*`, `CONF-F*`.

D13. Interfaz (front)

Decisión	Por qué
API por parámetro: cada pantalla recibe el cliente HTTP y se prueba con una API falsa (`apiFalsa.js`)	Mismo principio que el backend; las pruebas no dependen de red.
Vitest + React Testing Library en lugar de Jest	Vitest es el ejecutor natural de Vite, con la misma API que Jest. *Cambio respecto de lo planeado, avisado al grupo.*
Liquid Glass con `liquid-glass-react`, encerrada en un componente (`Vidrio.jsx`)	La exigencia es estética; si la librería falla o se reemplaza, se cambia un archivo. La paleta (`386641 · 6A994E · A7C957 · F2E8CF · BC4749`) y Lato las controla el CSS.
Límites de la librería	Se centra sola (rehecho como capa detrás del contenido) y su mapa de refracción es cuadrado: solo se usa en elementos con proporción ≤ 1,6 (`VID-11/12`); el resto usa el vidrio esmerilado del CSS. En Safari y Firefox la refracción no se ve y queda el esmerilado.
Accesibilidad	`prefers-reduced-transparency` (panel sólido), `prefers-reduced-motion`, foco visible, objetivos táctiles de 46 px, `aria-invalid` en los campos con error.
Calendario del PO en el front	Solo se pueden elegir hoy y mañana (ventana de 24 h); lunes, 25/12 y 1/1 aparecen cerrados. La regla real sigue en el backend.
Sesión en `localStorage` con `try/catch`	Conserva el acceso al recargar; si el navegador lo bloquea, la app funciona igual.
Mensajes en español por motivo (`mensajes.js`)	La API devuelve motivos, no textos (D1).

D14. Estilo de código y herramientas

PEP 8 con `ruff` (check + format, línea 100); nombres de pruebas en español, en `snake_case` y sin tildes, que nombran el comportamiento (`test_no_se_inscribe_sin_aceptar_los_terminos`). Un archivo de pruebas por grupo. Los datos de prueba comunes están en `datos_de_prueba.py` y los dobles en `dobles.py`.

D15. Flujo de trabajo en git

Una rama por tema (`feature/…`, `docs/…`), integradas a `main` con *pull request* y commit de merge. Los commits siguen `tipo(ámbito): rojo|verde|refactor – qué`. El historial se agrupa por tema (§2.1) para que cada ciclo se lea sin ruido.

5. Trazabilidad con la User Story

Criterio de aceptación / prueba de usuario	Dónde se resuelve	Pruebas
Elegir una actividad de la lista **con cupo**	`Actividad`, `disponibilidad`, `GET /actividades`	`ACT-*`, `CAT-04..09`, `ACT-F*`
Elegir el horario entre los **disponibles** (parque abierto, actividad disponible)	`Parque`, `Horario`, turnos del día	`PAR-*`, `HOR-*`, `HOR-F*`
Indicar la **cantidad** de personas	`Inscripcion` (máximo 10 y ≤ cupo restante)	`INS-*`, `VIS-F*`
Datos de cada visitante (nombre, DNI, edad, **talla si la actividad la pide**)	`Visitante`	`VIS-01..18`, `VAL-F*`
Aceptar los **T&C de la actividad**	`TerminosYCondiciones`, `terminos_texto`	`INS-*`, `TYC-*`, `CONF-F*`
Email con código QR al finalizar	`mensaje_email`, `qr`, SMTP	`QR-*`, `EML-*`, `SMTP-*`, `SRV-*`
Pasa: inscripción completa con email	`InscripcionService`	`ACE-01`
Falla: sin cupo en el horario	`Actividad`, repositorio	`ACE-02`, `REP-05`
Pasa: sin talla en una actividad que no la requiere	`Visitante`	`ACE-03`, `VIS-*`
Falla: parque cerrado o sin turno	`Parque`, `Horario`	`ACE-04`, `PAR-*`
Falla: sin aceptar los T&C	`Inscripcion`	`ACE-05`
Falla: sin la talla requerida	`Visitante`	`ACE-06`
Falla: T&C de otra actividad / sin haber iniciado sesión	`Inscripcion`	`ACE-07`, `ACE-08`

6. Estrategia de pruebas

6.1 Pirámide

- **Base (283 + 108):** unitarias del dominio, del servicio con dobles y de las pantallas con API falsa. Corren en segundos, sin infraestructura.
- **Medio (35):** contratos contra PostgreSQL real y `qrcode` real, incluida la carrera por el último cupo.
- **Cima:** aceptación (`ACE-01..08`, que además de las pruebas de usuario de la consigna cubren T&C ajenos y falta de sesión), cada una ejecutada con dobles **y** con PostgreSQL + QR real.
- **Externa (pendiente):** un email real a una casilla del grupo.

6.2 Contratos que se ejecutan dos veces

Repositorio de inscripciones, repositorio de usuarios y generador de QR comparten pruebas entre el doble y la pieza real. Es la garantía de que reemplazar un doble no cambia el comportamiento.

6.3 Hallazgos que encontraron las pruebas y las corridas manuales

#	Hallazgo	Quién lo encontró	Arreglo
1	El fake reservaba el cupo sobre una copia de la actividad	prueba de contrato `REP-03`	reservar sobre la del repositorio
2	Una prueba dependía de que el fake devolviera la misma instancia	`REP-04`	preparar el estado solo con la API del repositorio
3	La API se colgaba con credenciales SMTP de ejemplo	corrida manual	tiempo límite de 10 s (`SMTP-06`)
4	Sin sesión, el rechazo revelaba qué DNI estaban inscriptos	corrida manual	`USUARIO_NO_REGISTRADO` temprano (`SRV-23`)
5	T&C de una actividad inexistente daba error 500	`SRV-30`	404
6	La API rechazaba al front abierto en `127.0.0.1` (CORS)	corrida manual en el navegador	orígenes configurables (`MAIN-05/06`)
7	El vidrio de la librería dibujaba un contorno interior en elementos anchos	corrida manual en el navegador	proporción máxima 1,6 (`VID-11/12`)
8	Una regla nueva (`INS-11`) rompió `INS-02` (actividad nula)	la suite completa	guarda en `errores_por_visitante`

7. Límites y trabajo futuro

Tema	Límite actual	Mejora posible
Caída entre guardar y enviar el email	La inscripción queda guardada sin email (D6)	Estado *pendiente* + reintentos en segundo plano
Cancelaciones	No hay cancelar ni modificar una inscripción (no está en la US)	Nueva US
Sesión	Token en `localStorage`, sin renovación	Cookie `HttpOnly` y refresco
Contraseña	Mínimo de 6 caracteres (*provisorio*)	Definir la política con el PO
Menores de edad	Se pide la edad pero no hay un adulto responsable	Definirlo con el PO (P10b)
Refracción del vidrio	Solo en navegadores Chromium	Evaluar otra técnica
Preguntas de baja prioridad	P10b, P12b, P15b, P18, P19b, P20, P21 siguen abiertas	Resolver con el PO

8. Cómo ejecutarlo

Los pasos completos están en `codigo/README.md`. En resumen:

```
cd codigo
docker compose up -d                        # PostgreSQL
cd backend
PYTHONPATH=src .venv/Scripts/python.exe -m ecoharmony.preparar_base
.venv/Scripts/python.exe -m pytest          # 318 pruebas (las de base se saltean si no hay
PostgreSQL)
.venv/Scripts/python.exe -m ruff check .    # PEP 8
cd ../frontend
npm install && npm test                     # 108 pruebas
```

9. Bibliografía

- Beck, K. *Test-Driven Development by Example*. Addison-Wesley, 2002. Patrones: *Starter Test*, *One Step Test*, *Fake It*, *Triangulate*, *Obvious Implementation*, *Child Test*, *Mock Object*, *Crash Test Dummy*, *Clean Check-in*.
- Ble, C. *Diseño Ágil con TDD*. Caps. 3 (ATDD), 6 (dobles de prueba), 8–9 (proyecto de ejemplo).
- PEP 8 – *Style Guide for Python Code* (estilo elegido por el grupo en el TP2).
- Enunciado del TP2 (EcoHarmony Park) y respuestas del Product Owner (ver preguntas al PO).

10. Puntos que el grupo debe confirmar antes de entregar

1. **Textos de T&C** (borrador): clima y suspensión de la actividad, menores de edad / adulto responsable, redacción de datos personales, y las recomendaciones propias de cada actividad (calzado, accesorios, uso de flash).
2. **Prueba externa de email real** (§2.4): falta cargar credenciales SMTP en el `.env` y ejecutar la prueba de categoría X.
3. **Preguntas de baja prioridad al PO** (§7).